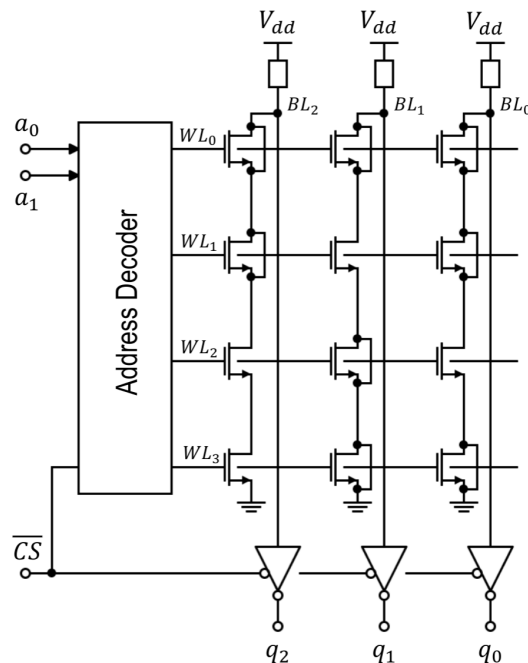


1 Lesson 07-10-26

1.0.1 NAND memory

In the last lesson we saw *NOR memories*. Now we look at the **NAND** structure.



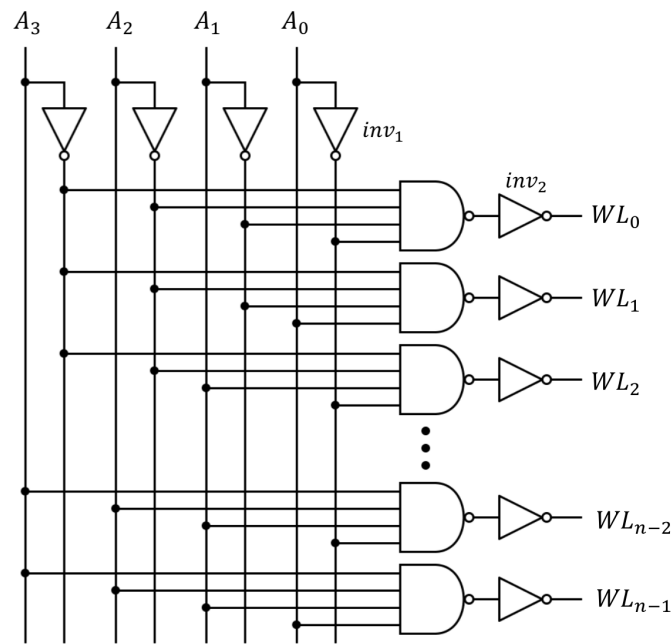
All bit lines are normally pulled-up to V_{dd} : the output is still logic 0 because of the inverter.

Now, we know that if all transistors are at gate low, we don't connect them to ground: it follows that the bit line stays high and the output low (logic 0 at default).

The control signals for the word lines (WL_i) are active low. This means that every word line is at 1 but the selected one (which is at 0).

Now:

- If there is no short circuit between the drain and source of the selected transistor, we don't get a short to ground, and the line stays high: so the output stays low;
- If there is a short circuit, on the other hand, we short to ground, meaning the output stays high: we encode a logic 1 by shorting a transistor.



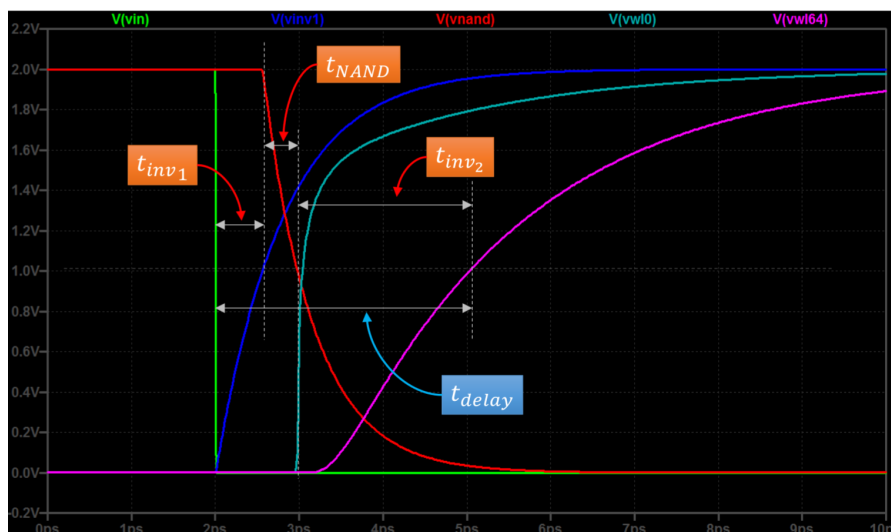
Clearly, the mask topology of the NAND topology is more complicated. We have the delay of the address decoder, given by:

1. Delay of the address lines inverters, which are driving half of the NAND gates, and have large parasitic capacitance.
2. Delay of the NAND gates, with large parasitic resistance when all pull-down transistors are ON;
3. Lastly there's the delay of the Word Line inverters, driving all transistors in the bit cells in a row, with large parasitic capacitance and resistance as the word Line is long.

This tradeoff comes at the gain of higher *densities* overall, meaning NAND memories are usually less performant but smaller.

1.0.2 Memory timings

Let's look at the simulated delay, specifically of the address decoder.



We see the 3 delay source cited above (t_{inv1} , t_{NAND} , and t_{inv2}). The total delay is:

$$t_{delay} = t_{inv1} + t_{NAND} + t_{inv2}$$

Note that this is the example with 64 bit cells in a row: the delay for the first bit cell is much lower than the delay of the last bit cell. We must always consider the worst case!

We must also look at the performance of NOR and NAND technology on bit line discharging. The bit line has a certain parasitic capacitance C_L . When the memory is idle, the bit line is pulled-up to V_{dd} , and C_L charges up to it. When a bit is read, we have to discharge C_L down to 0.

- In a **NOR** structure, the parasitic capacitance dominates. The resistance of the bit line is small (metal), and all capacitances are in parallel. This means that:

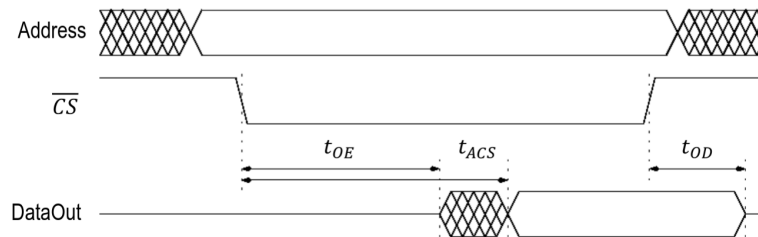
$$\tau = R_{DSon} \cdot C_L = R_{DSon} \cdot C_{drain} \cdot n_{tr}$$

where n_{tr} is the number of transistors;

- In NAND topology the parasitic resistance dominates: it is the channel resistance of all transistors in the series. The structure can be studied using the *Elmore* delay, and we get that:

$$\tau = \frac{RC}{2}, \quad R = n_{tr} \cdot R_{DSon}, \quad C = n_{tr} \cdot C_{drain}$$

Latly, we show a picture of the general timing parameters for a memory access (a read, as we're talking EPROM memories) operation.

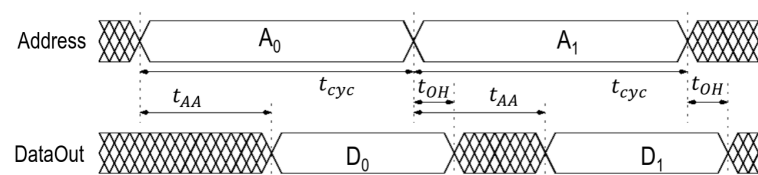


From when address is stable and $\overline{CS}$ (active low) is low, we note the following delays:

- t_{OE} is the **minimum Output Enable** time, which is the time before the output takes some value, but is not yet stable. After t_{OE} the value on DataOut might be wrong, but DataOut is not in high impedance anymore;
- t_{ACS} is the **maximum Chip Select Access** time, after which the DataOut will be stable;
- t_{OD} is the **maximum Output Disable** time, after which the output returns to high impedance.

We then see that the time we have to wait before having a result from memory is in between t_{OE} and t_{ACS} .

Let's see what happens when changing the address.



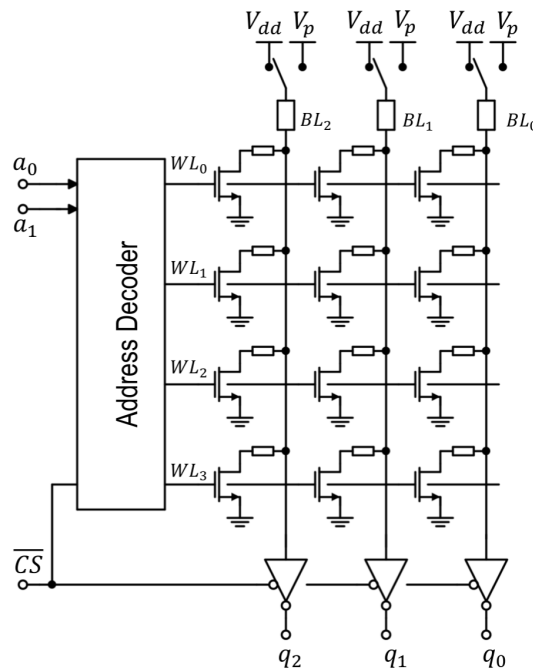
From when the address is changed, we note the following delays:

- t_{AA} is the **maximum Address Access** time, after which DataOut takes the value pointed to by that address;
- t_{OH} is the **minimum Output Hold** time, after which the DataOut goes back to being undetermined after address changes;
- t_{cyc} is the **cycle** time, for which addresses should at least be stable.

We then see that, as long as we keep addresses stable for at least t_{cyc} , the data will be available at most after t_{AA} , and at least for t_{OH} after the address change.

1.1 One Time Programmable Read Only Memories

We now look at **OTP-ROM** (*One Time Programmable Read Only Memories*). These are based on **fuses** or *anti-fuses* (which blow to become shorts), which get blown in order to encode data. Of course this process is irreversible and means writing operations are permanent.



As in the standard NOR structure, all bit cells have a transistor, with the drain of each transistor is connected to the bit line through a fuse.

All fuses are short circuit after fabrication, meaning all transistors are connected to the bit lines: the initial content of the memory is all 1s.

Blowing a fuse turns a logic 1 into a logic 0, with the corresponding transistor being disconnected from the bit line.

As we said, the fuse cannot be restored, the operation is one time and we cannot turn a logic 0 back to a logic 1.

To correctly program an OTPROM the procedure is to:

- Enable the word line of the bit we want to program;
- Select the bit line of the bit we want to program;
- Drive the bit line at $V_p \gg V_{dd}$, high enough to blow the fuse.

We call the mode in which V_p is feed to the memory *programming mode*. In programming mode the data bus is an input which selects the bit lines to be connected to V_p . V_p itself is provided through a dedicated memory pin.

1.1.1 Performance of OTPROMs

First off, **writing** an OTPROM takes considerably (orders of magnitude) higher time than reading it. This is fine, as writing an OTPROM is as we said an one-time only operation.

As for other characteristics, we see that a **read** operation is mostly identical in performance to Mask Programmable ROMs. The disadvantages are higher resistances of the fuse compared to a short circuit, which get a bit slower compared to a Mask Programmable ROM.

OTPROMs are also *unntestable*: they need to be produced in small size to have a good yield.

The number of defective ICs in a wafer does not depend on the IC size, but smaller ICs of course have less chance to be defective. This means that usually there are more small ICs in a wafer than large ICs. Note that the yield y is roughly given by:

$$y = \frac{n_{ICok}}{n_{ICtot}} = 1 - \frac{n_{ICdefective}}{n_{ICtot}}$$

1.2 Programmable Read Only Memories

OTPROMs are suboptimal as they can be written *only once*. Because of this, more advanced solutions which allow memory to be rewritten (that is, *reprogrammed*), a number of times. We call these **PROMs** (*Programmable Read Only Memories*).

1.2.1 Floating-gate transistors

The core element of a PROM is a *floating-gate* transistor, that is a MOS transistor which can have its gate biased by applying a high voltage to it. One example of such a transistor is the **FAMOS** (*Floating gate Avalanche MOS*), which is actually not famous at all.

They are similar to a MOS transistor, with an additional polysilicon layer between two thin SiO_2 layers below the gate. This means the gate is *floating*. Charges trapped in the floating gate change the threshold voltage V_T , where

- Negative charges increase the threshold;
- Positive charges decrease the threshold.

Programming

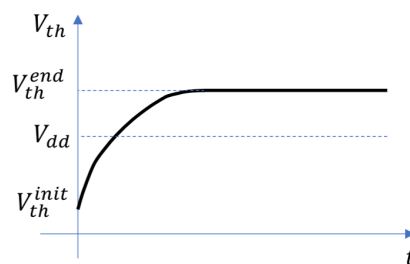
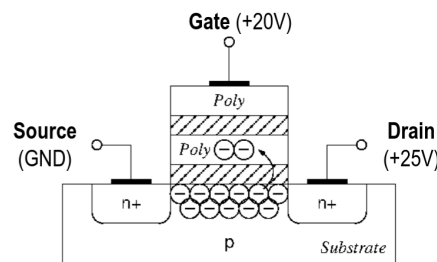
The floating gate is completely isolated, and to program it we use *hot electron programming*:

- By applying a high programming voltage V_p to the gate, a rich channel forms (as the gate is pulled high). This is not enough to lower V_T !
- By applying a high programming voltage V'_p to the drain-source channel, electrons in the channel are subject to strong electrical fields. Due to strong vertical electrical field from the gate, the high energy electrons may overcome the SiO_2 barrier, reach the floating gate. The result is that the gate is pulled low and V_T increases.

Performance

When programming a FAMOS transistor, as the threshold voltage V_T increases, the channel is less rich and the vertical electrical field is less strong. This clearly means that the number of electrons with enough energy to jump in the floating gate decreases (less electrical field), and process is *self regulating* (!) The nice property of this system is that the final threshold voltage is quite consistent among all transistors, regardless of the length of the process.

We then have that V_T grows above V_{dd} by a certain margin. By playing with doping levels, densities, geometries, we can change that margin.



The *disadvantages* of FAMOS transistors are:

- The process takes time and requires high voltages
- How to generate those voltages in the first place: provided externally using pins? Generated internally from the standard voltage supply?
- Damages to the lattice: we cannot apply programming too many times. 1000 – 10000 programming cycles seems to be the sweet spot in most commercial applications;
- Leakage current removes electrons from the floating gate: the information is lost over time, on a timescale of several tens of years.

Erasing

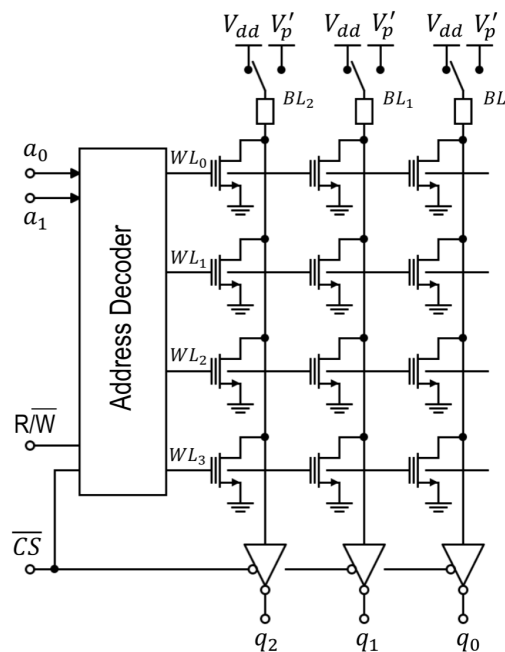
Erasing FAMOS transistors is the most interesting part. Electrons cannot be removed electrically from the floating gate: energy has to be provided to the electrons via an *UV lamp*.

As high energy photons hit electrons, the electrons jump back to the substrate and diffuse. After a sufficiently long amount of time (minutes) all electrons are removed, and the floating gate is left unbiased.

For this reason we need a window on the integrated circuit (which we'll tape off when the circuit is at work to avoid accidental exposure to UV light). We also cannot target a *single* transistor: all transistors are erased at the same time. This makes PROM erasure an usually out of the circuit operation, as with programming, done with external hardware in a lab.

1.2.2 Programmable ROM

Let's look at a NOR structure **PROM**.



The funky transistor symbol with the triple gate line represents a FAMOS transistor (with the middle line being the floating gate).

All bit cells have a FAMOS transistor. The drain of each transistor is connected to the bit line. All FAMOS transistors are not programmed after fabrication, so they behave normally. The initial content of the memory, as a result, is all 1s, and programming a transistor (raising the V_T higher, disconnecting it) turns a logic 1 into a logic 0.

As we said, the memory can be reset to the initial state by shining UV rays through a window. All programmed transistors are erased, and all non programmed transistors are not affected: the memory contents return to all 1s.

To correctly program a PROM we proceed similarly to OTPROMs:

- Enable the word line of the bit we want to program;
- Select the bit line of the bit we want to program;
- The selected word line is driven to the voltage $V_p > V_{dd}$;

- The selected bit lines (possibly more than one for the same word) are driven to the voltage $V_p' > V_{dd}$.

To avoid impacting other components, other rows are at ground, and bits not to be programmed on this row have their drains at V_{dd} .

1.2.3 Performance of PROMs

For the read operation, PROMs are identical to Mask Programmable ROM and OTPROMs.

For the *write* operation, that is *programming*, moving negative charges to the floating gate makes the transistor a permanent open circuit:

- Turns a 1 into a 0 (if the bit line is inverted);
- Turns a 0 into a 1 (if the bit line is affirmed).

It works at the word level too.

For erasing, we remove the negative charges from the floating gate. This restores the normal transistor behavior, but applies to the entire memory at once.

This means PROMs have almost completely superseded OTPROMs:

- Much more flexible;
- Cheaper during prototyping;
- Can be reused;
- Still, not suitable for data that is frequently changed.

1.2.4 FLOTOX transistors

Another variant of floating-gate transistor is the **FLOTOX** (*FLO*ating gate *T*unneling *O*Xide transistor). Similarly to FAMOS, it has a floating gate.

The FLOTOX differs from the FAMOS in its erasure properties: it is *electrically erasable*.

This is achieved through the *tunneling effect*. The tunneling effect is a quantum mechanical phenomenon, where a particle (electron) can pass a potential barrier even if it has insufficient energy.

A thinner oxide layer on the drain (source) side makes tunneling easier. Electrons come from the drain (source) region rather than from the substrate or channel. An higher electrical field is necessary for this.

Programming

The floating gate is completely isolated, and to program it we use *Fowler–Nordheim tunneling*:

- By applying a high programming voltage V_p to the control gate, a strong electric field is established across the thin tunnel oxide.
- The strong electric field allows electrons to tunnel through the thin SiO_2 barrier from the substrate onto the floating gate.
- The floating gate accumulates negative charge. As a result, the threshold voltage increases:

$$Q_{FG} < 0 \quad \Rightarrow \quad V_T \uparrow .$$

To erase the cell, the polarity of the applied voltage is reversed:

- A high voltage of opposite polarity is applied to the control gate, producing a strong electric field across the tunnel oxide.
- Electrons tunnel from the floating gate through the thin SiO_2 layer into the substrate.
- The floating gate loses its negative charge, causing the threshold voltage to decrease:

$$Q_{FG} \rightarrow 0 \Rightarrow V_T \downarrow.$$

Performance

Let's make a comparison of programming mechanisms:

- Hot Electron Programming is faster than Tunneling, which may increase performance, but performance may be limited by other issues.
- Tunneling causes less damages to the lattice than Hot Electron Programming, leading to higher life-time of the device, and many more programming cycles;
- Hot Electron Programming is Self Regulated, Tunneling is not;
- Tunneling requires careful timing to achieve good consistency: the resulting distribution of V_T after programming has a larger standard deviation, and requires larger margins during operations;
- FLOTOX transistors are larger and harder to fabricate: lower yield, higher costs.

Addenda on programming

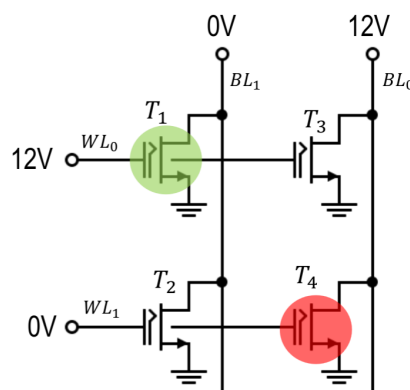
There is a famous issue with FLOTOX programming. The straightforward implementation (but it doesn't work!) is:

- The drain of each transistor is connected to the bit line;
- The source of each transistor is grounded.

This has a problem: programming a transistor erases another one.

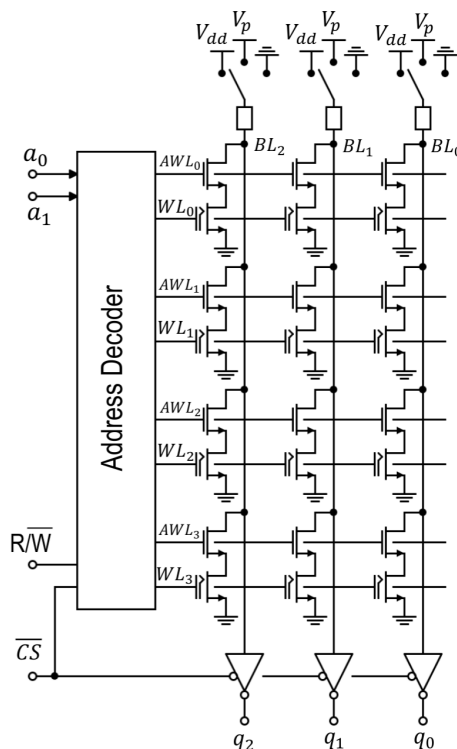
- To program T_1 : Gate: $WL_0 = 12\text{ V}$, Drain: $BL_1 = 0\text{ V}$
- To avoid programming T_2 : $WL_1 = 0\text{ V}$
- To avoid programming T_3 : $BL_0 = 12\text{ V}$, $V_{GD} = 0$
- However, T_4 is erased: $WL_1 = 0\text{ V}$, $BL_0 = 12\text{ V}$

The solution is to use an access transistor for each bit cell.



1.2.5 Electrically Erasable Programmable ROM

We use FLOTOX transistors to realize **EEPROMs** (*Electrically Erasable Programmable ROMs*).



All bit cells have a FLOTOX transistor, with thin oxide on the drain side.

All bit cells also have an access transistor; drain connected to the bit line, source connected to the FLOTOX, gate driven by the address decoder (via dedicated AWL_i lines).

Access transistors are activated when the word is selected.

Out of the factory, all transistors behave normally, and the initial content of the memory is all 1s.

- Programming a transistor turns a logic 1 into a logic 0;
- Erasing a transistor turns a logic 0 back into a logic 1.

This happens *electrically* and *in circuit*.

Let's look at how the main operations are carried out:

- For **programming**, we select the word by raising the word line, and the data in the word by raising the D line (which as always acts as an input for programming).

We then enter the *programming* mode by enabling AWL on the word line, that is enabling the access transistors. We keep these OFF on other lines to avoid affecting other FLOTOX transistors.

At this point, to program we raise the word line (that is the gate) to V_p , and the selected bit lines (that is the drains) to 0 V. The rest of the lines are kept at V_p ;

- For **erasing**, we select the word by raising the word line, and the data (to erase) in the word by raising the D line.

We then enter the *erasing* mode by enabling AWL on the word line, that is enabling the access transistors. We keep these OFF on other lines to avoid affecting other FLOTOX transistors.

At this point, to erase we lower the word line (that is the gate) to 0 V, and the selected bit lines (that is the drains) to V_p . The rest of the lines are kept at 0 V;

1.2.6 Performance of EEPROMs

The read operation is identical to Mask Programmable ROM, OTPROM and EPROMs (which we called PROMs so far). The external interface is to:

- Provide programming voltages;
- Signal to select programming;
- Signal to select erasing.

External interface often simplified by providing internal programming and erasing mechanisms.

They are widely used, and much more flexible than EPROMs, as they can be erased by a micro-processor in circuit. On the other hand, they are expensive, and the bit cell is larger than in an EPROM.

Writeability is very good as it is:

- In circuit;
- Repeatable, thousands of times.

We also have *erasure* in circuit, with single bit access both in writing and erasing.

The storage permanence is very high, and the NREs are kept low (though they are higher than EPROMs).